

Impianti di Elaborazione

Antonio Sirignano*,a,1

^aUniversità degli studi Federico II

This manuscript was compile on 2 ottobre 2026

Sommario

Progetto per l'esame di Impianti di Elaborazione.
GitHub Link: <https://github.com/antsiri/Progetto-di-Impianti-di-Elaborazione->
Keywords: Capacity Test

INDICE

1	Introduzione	1
2	Capacity Test	1
2.1	Setup esperimento	1
2.2	Descrizione dei test	1
2.3	Grafici e analisi	2
2.4	Analisi della Fairness e della Ripetibilità del Benchmark	2
3	PCA e Clustering	3
3.1	Obiettivo e Setup	3
3.2	Raccolta dati	3
3.3	Pulizia e caratterizzazione HL/LL (reale)	3
3.4	Clustering e scalabilità	5
3.5	Costruzione del workload sintetico	5
3.6	Caratterizzazione LL (sintetico)	6
3.7	Validazione Statistica	6
3.8	Conclusioni	7
4	Workload Characterization	7
5	DoE	7
6	Regression	7
7	Benchmark	7
8	FFDA	7
9	RDB e Fault Tree	7

1. INTRODUZIONE

In questa documentazione vengono spiegati e risolti problemi legati agli Impianti di Elaborazione.

2. CAPACITY TEST

2.1. Setup esperimento

Lo scopo del seguente esperimento p quello di utilizzare la **mas-sima capacità** di un Web Server.
A tale scopo, è stata impostata una piattaforma sperimentale su **mac-china virtuale** che ospita un sistema operativo **Linux Ubuntu 20.04**, il quale a sua volta ha in esecuzione un servizio **Apache2** che funge

da gestore del web server.
Per simulare i processi di richieste di un determianto **carico** sul web server, vengono generate dei file di varia dimensione utilizzando il co-mando `allocate`. Le dimensioni variano da pochi kiloByte fino a cento MegaByte.
La **caratterizzazione della capacity** è un test che richiede un certo numero di queste risorse per valutare la stabilità del sistema a fronte di un crescente tasso di richieste.
Le specifiche tecniche della macchina virtuale su cui è ospitato il web server sono le seguenti:

Tabella 1. Caratteristiche della Macchina Virtuale (UTM)		
Categoria	Parametro	Valore
Sistema	Architettura	x86_64
Sistema	Macchina Emulata	Standard PC (Q35 + ICH9, 2009)
Sistema	Sistema Operativo	Ubuntu 22.04.5 LTS
Hardware	Memoria RAM	10 GB
Hardware	Dimensione Disco	25,75 GB
Rete	Tipologia Rete	Shared Network (e1000)

Note: Dati rilevati dal gestore di macchine virtuali UTM su host Mac.

Per l'esecuzione dei test, è stato utilizzato **Apache JMeter 21.0.12**, eseguito sulla macchina Host con le seguenti specifiche:

Tabella 2. Caratteristiche della Macchina Host (MacBook Pro)		
Categoria	Parametro	Valore
Hardware	Modello	MacBook Pro (16-inch, 2019)
Hardware	Processore	2,3 GHz 8-Core Intel Core i9
Hardware	Memoria RAM	16 GB 2667 MHz DDR4
Hardware	Scheda Grafica	AMD Radeon Pro 5500M 4 GB
Hardware	Archiviazione	1 TB (Macintosh HD)
Sistema	Sistema Operativo	macOS Tahoe (v26.6)

Note: Dati rilevati dalle impostazioni di sistema della macchina Host.

2.2. Descrizione dei test

Il test si compone di un singolo **Thread Group**. Il gruppo ha la seguente configurazione:

- `Number of Threads (users)`: 50
- `Ramp-up Period (seconds)`: 25

^{*}Corresponding author: ant.sirignano@studenti.unina.it (A. Sirignano).
[†]These authors contributed equally to this work.

This document class was prepared on VSCode and compiled with pdfTeX. No errors were found during the compilation process. Rho-class supports external editors, though additional setup may be required.

- **Duration (seconds):** 300

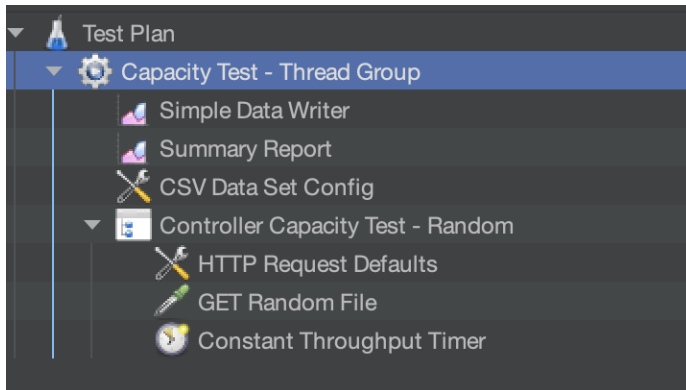


Figura 1. JMeter Test Plan.

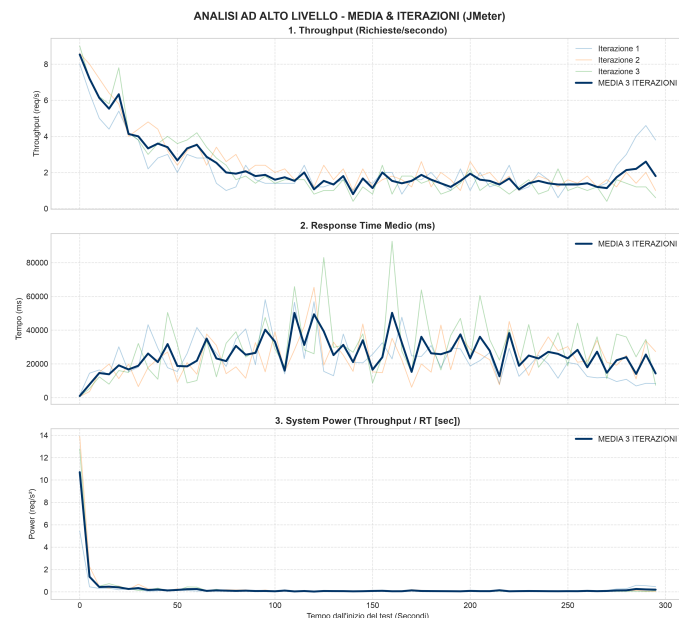
Nel test è stato utilizzato uno script che genera file di diverse dimensione sul server. Al fine di tenere traccia dei file generati, viene scritto un file `.csv` che contiene il path di ogni singolo file. Per questo nel test vi è un **CSV Data Set Config**, il quale legge ogni riga corrispondente al filepath di ogni file generato.

All'interno del gruppo è presente un **Random Controller**, in cui è inserita un richiesta HTTP di risorse. Il **Constant Throughput Timer** permette di regolare il numero di richieste da effettuare al minuto. Al fine di ottenere dei report esplicativi dell'esecuzione del test, è stato utilizzato un **Simple Data Writer**.

Per le informazioni a basso livello (analisi del bottleneck di prestazioni) è stato utilizzato il tool **vmstat**, fatto eseguire sul server.

Infine i dati ottenuti vengono processati attraverso uno script Python.

2.3. Grafici e analisi



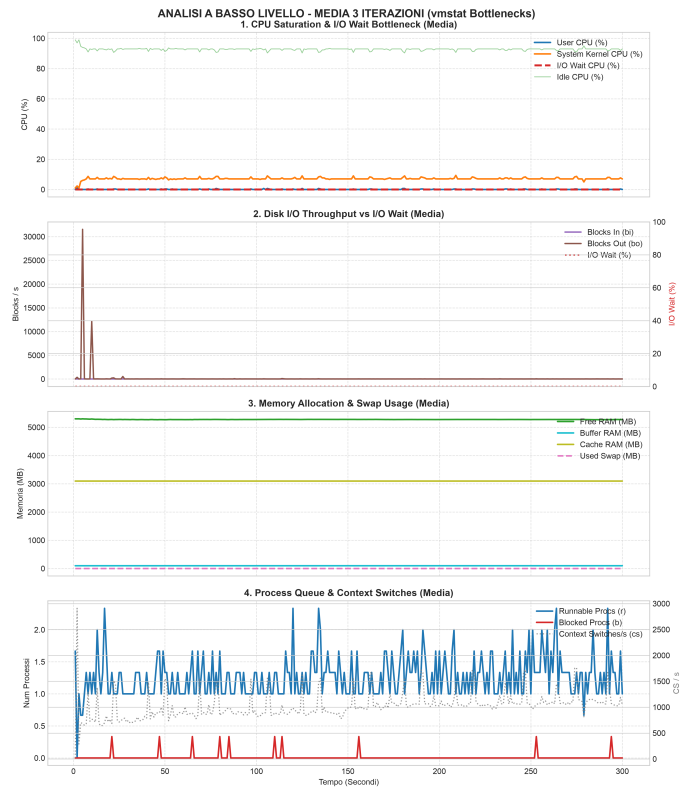
Eseguito lo script, sono stati ottenuti i seguenti grafici, che rappresentano rispettivamente il **throughput**, ovvero il tasso di richieste che il sistema ha potuto servire, il **response time**, ovvero la media dei tempi di risposta delle richieste per quel carico, e la **power**, che è un valore riassuntivo dei precedenti due parametri, ovvero

$$\text{Power} = \frac{\text{Throughput}}{\text{Response Time}}.$$

(1)

Analizzando tale grafico si può notare che:

- Il Throughput crolla rapidamente da 8 a ~1.5 req/sec.
- Il Response Time esplode fino a 20-50 secondi.
- Il Power crolla a zero nei primi secondi: il sistema perde totalmente la sua efficienza di elaborazione.



Analisi dei bottleneck Analizzando i dati a basso livello, si vede che:

- La CPU è quasi totalmente in attiva (~92)
- La RAM libera è maggiore di 5GB.
- Il Disk I/O e I/O Wait sono piatti dopo il picco iniziale.

Questo fa pensare che il collo di bottiglia non è di tipo hardware.

Si tratta invece di un bottleneck di configurazione software: i worker/thread di Apache sono saturi per via dei file grandi in download, mettendo le nuove richieste in coda senza che la CPU debba effettivamente lavorare.

2.4. Analisi della Fairness e della Ripetibilità del Benchmark

Per valutare l'equità nella ripartizione delle risorse di rete e la stabilità complessiva dei test di carico, è stato adottato l'**Indice di Fairness di Jain (Jain's Fairness Index)**.

La valutazione è stata condotta su due livelli: inter-iterazione (per verificare la riproducibilità e determinismo del test) e intra-iterazione (per rilevare eventuali fenomeni di starvation tra i thread/client concorrenti).

Dato un insieme di n entità concorrenti e definita con x_i la metrica di prestazione associata all'entità i -esima, l'indice Jain J è calcolato come:

$$J(x_1, x_2, \dots, x_n) = \frac{(\sum_{i=1}^n x_i)^2}{n \sum_{i=1}^n x_i^2} \quad (2)$$

L'indice assume valori compresi nell'intervallo $[\frac{1}{n}, 1.0]$.

Poiché i test prevedono l'esecuzione di Thread Group Paralleli che

richiedono risorse di dimensioni differenti, la metrica, x_i non misura il semplice numero di richieste al secondo, che premierebbe fittiziamente i file di piccola dimensione, bensì la **banda passante occupata** espressa in MB/s:

$$x_i = \text{Bandwidth}_i = \frac{\text{Byte Total Trasferiti}_i}{1024^2 \cdot \Delta t} \quad (3)$$

L'indice di Jain inter-iterazione calcolato sia sul Throughput (0.9965) sia sulla Bandwidth (0.9968) è eccezionalmente prossimo al valore ideale di 1.0. Questo risultato attesta che il setup di test è perfettamente deterministico, stabile e riproducibile. Non sono state rilevate interferenze esterne (es. rumore di rete, processi di background sulla VM) che abbiano alterato i risultati tra un'esecuzione e l'altra.

Analizzando i dati all'interno della singola iterazione, l'indice di Jain calcolato sui thread scende sotto la soglia ottimale (0.90), oscillando tra 0.7441 e 0.8163. Un valore di fairness significativamente inferiore a 1.0 all'interno della singola esecuzione dimostra l'insorgere di Thread Starvation. Sotto il carico massimo di 50 utenti concorrenti, le risorse di elaborazione del server web non vengono distribuite in modo uniforme: alcuni thread riescono ad accaparrarsi i socket di rete e completare più richieste, mentre altri rimangono bloccati nelle code del server per tempi prolungati (spiegando i tempi di risposta massimi fino a 200+ secondi registrati da JMeter).

3. PCA E CLUSTERING

3.1. Obiettivo e Setup

L'obiettivo è quello di effettuare una **workload characterization** completa: generando un workload reale su un Web Server, si effettueranno misurazioni HL e LL, applicando PCA e clustering su entrambe. Si effettua inoltre la costruzione di un workload sintetico e si effettua un confronto statistico tra i due carichi. Per effettuare ciò viene utilizzata la macchina già precedentemente descritta nella tabella 2.

3.2. Raccolta dati

3.2.1. Generazione dei file campione

Per la generazione dei file campione, viene utilizzata la logica delle 5 categorie di taglia del documento (Low, Mid-Low, Mid, Mid-High, High), ma con dati casuali piuttosto che file reali multimediali.

Per semplicità, viene istanziato sulla stessa macchina un Web Server, utilizzando `FastAPI`.

3.2.2. Generazione del traffico e misurazione

Una parte fondamentale ora è la raccolta di dati statistici durante l'esecuzione delle richieste al Server. Utilizzando file statici, vengono effettuate delle richieste al server dei file campione.

- **Load Generator:** genera traffico HTTP concorrente verso le risorse, con un numero configurabile di thread worker e un ramp—up graduale (i thread partono scaglionati nel tempo, non tutti insieme). Le richieste sono pesate per categoria dimensionale (`DEFAULT_TIER_WEIGHTS`), in modo da avere un traffico quanto più realistico possibile.
- **Collector:** basato su `psutil`, raccoglie due tipi di metriche a intervalli regolari
 - globali di sistema (CPU, memoria, swap, I/O disco/rete — queste ultime come snapshot delta tra snapshot successivi, non contatori cumulativi da boot, altrimenti la PCA noterebbe solo il tempo che scorre)
 - di processo, specifiche del server (CPU%, RSS, thread attivi, file descriptor aperti)

Tabella 3. Ripartizione dei File per Fascia di Risorse (RESOURCE_TIERS)

Categoria	Parametro	Valore
low	File 1	13,5 KB
	File 2	20 KB
	File 3	23,4 KB
	File 4	98 KB
	File 5	99,7 KB
mid_low	File 1	137 KB
	File 2	139 KB
	File 3	184 KB
	File 4	458 KB
	File 5	514 KB
mid	File 1	716 KB
	File 2	0,98 MB
	File 3	0,99 MB
	File 4	1 MB
	File 5	1,3 MB
mid_high	File 1	2 MB
	File 2	2,13 MB
	File 3	2,14 MB
	File 4	5,07 MB
	File 5	5,19 MB
high	File 1	9,21 MB
	File 2	9,38 MB
	File 3	9,58 MB
	File 4	10,1 MB

Note: Dati estratti dal dizionario Python `RESOURCE_TIERS`.

- **Orchestrazione:** i due processi (collector e load generator) vengono avviati in parallelo, non in sequenza, coordinati da uno script dedicato; infatti è essenziale che si sovrappongano temporalmente, poiché il `Collector` deve effettuare le misurazioni durante la generazione del carico. Tutti i thread worker sono attivi simultaneamente per l'intera durata del test.

3.3. Pulizia e caratterizzazione HL/LL (reale)

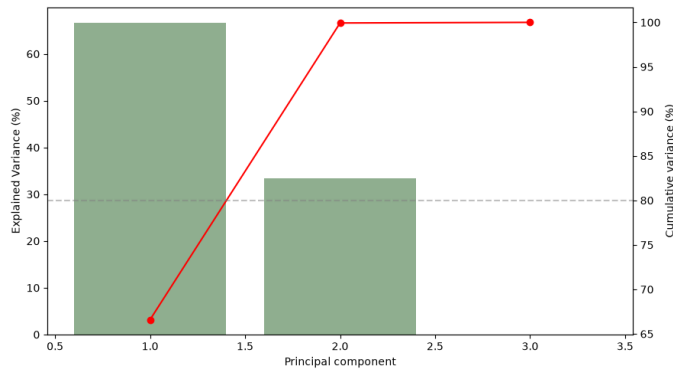
3.3.1. Pulizia del dataset

Prima di procedere con PCA e clustering, ciascun dataset (HL, LL globale, LL di processo) viene sottoposto a un'analisi preliminare delle distribuzioni campionarie. Per ogni colonna vengono calcolate la media, la deviazione standard e i quartili; inoltre, viene generato un istogramma affiancato da un boxplot al fine di individuare colonne poco significative o affette da outlier. In particolare, vengono identificate come candidate alla rimozione le colonne che presentano una cardinalità di valori unici estremamente bassa rispetto al numero di osservazioni (soglia arbitraria dell'1%), e le righe che si discostano dalla media di oltre 4 deviazioni standard su almeno una colonna. È importante sottolineare che questi criteri automatici forniscono solamente un *segnale*: la decisione finale di rimozione viene presa dopo l'ispezione manuale, per evitare di eliminare varianza informativa (ad esempio, colonne con pochi valori disitinti ma comunque rilevanti, come `swap_percent`).

Dataset LL globale. La colonna `mem_total` risulta costante per l'intera durata della misurazione e viene pertanto rimossa. Le colonne relative allo swap (`swap_used`, `swap_free`, `swap_percent`) presentano una bassa cardinalità di valori, dovuta dal fatto che il gestore di memoria del sistema operativo le aggiorna a blocchi discreti piuttosto che con continuità; tuttavia mostrando un range di variazione significativo, vengono mantenute. Dall'analisi degli outlier emerge un pattern ricorrente: un blocco di righe consecutive all'inizio della misurazione (transiente di *ramp-up*, corrispondente all'avvio graduale dei thread worker) e un blocco analogo al termine (transiente di *coll-down*, dovuto alla chiusura delle connessioni e alla fase di rientro del sistema a uno stato di riposo). Tali righe vengono rimosse, per un totale di 32 righe iniziali e 23 finali.

Tabella 4. Sintesi delle operazioni di pulizia applicate ai dataset reali

Dataset	Righe (grezze → pulite)	Colonne rimosse	Trimming
LL globale	3615 → 3560	<code>mem_total</code>	32 / 23
LL processo	3615 → 3560	<code>pid</code> , I/O	32 / 23
HL	679046 → 679046	<code>responseCode</code> , <code>success</code> , <code>URL</code>	—

**Figura 2.** Scree plot delle componenti principali — LL globale (reale).

Dataset LL di processo. . Vengono rimosse le colonne `pid` (costante) e `proc_io_read_bytes/proc_io_write_bytes`, non disponibili per la piattaforma utilizzata per la raccolta dati. Analogamente al caso precedente, vengono rimosse 32 righe iniziali e 23 finali corrispondenti ai transienti di avvio e chiusura.

Dataset HL. Vengono rimosse le colonne `responseCode` e `success` (costanti poiché tutte le richieste hanno avuto esito positivo) e `URL` (ridondante rispetto a `label`). A differenza dei dataset LL, non viene applicato alcun trimming per indice: gli outlier individuati (circa lo 0.3% delle osservazioni) risultano distribuiti lungo l'intera durata della misurazione, non concentrati in un transiente iniziale o finale, e vengono pertanto interpretati come variabilità fisiologica del sistema sotto carico piuttosto che errori di misura.

3.3.2. Analisi delle componenti principali (PCA)

L'analisi PCA viene condotta standardizzando preventivamente le feature (essendo le colonne espresse su scale eterogenee: percentuali, byte, conteggi), per poi analizzare la varianza spiegata cumulativa da ciascuna componente.

Per il dataset LL globale, si osserva che le prime 3 componenti principali spiegano oltre l'80% della varianza totale, coerentemente con l'attesa correlazione tra le metriche di memoria (`mem_used`, `mem_available`, `mem_free`) e quelle di I/O.

3.3.3. Clustering gerarchico e selezione dei parametri

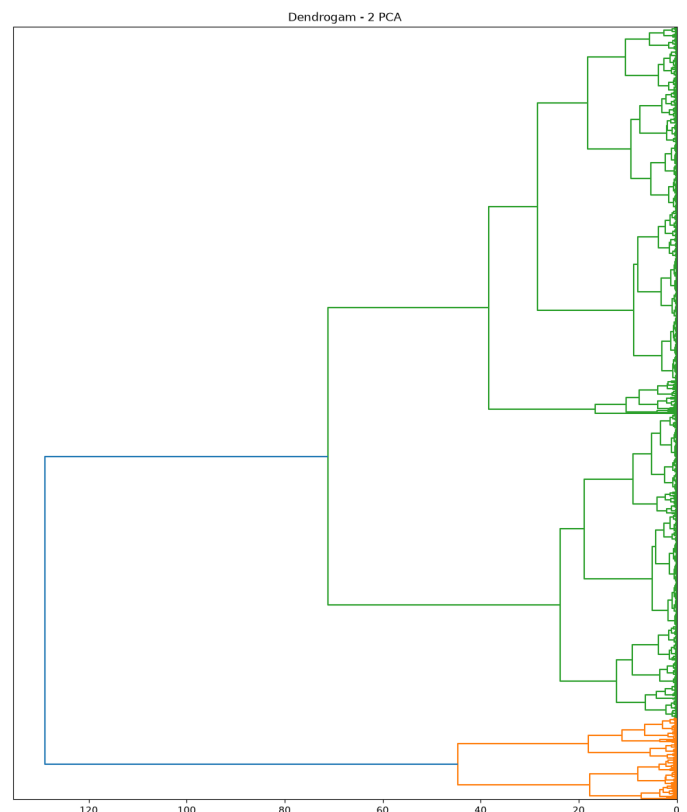
Per determinare la combinazione ottimale di numero di componenti principali e numero di cluster, viene condotto uno sweep sistematico: per ciascuna coppia (numero di componenti, numero di cluster) viene applicato un clustering gerarchico agglomerativo (metodo di Ward), calcolando la devianza intra-cluster, inter-cluster e la devianza totale persa.

Sulla base di questo criterio, per il dataset LL globale viene scelta la combinazione a **3 componenti e 15 cluster**; per il dataset LL di processo, la combinazione a **2 componenti e 15 cluster** (devianza persa pari a 0.0609).

Tabella 5. Sweep delle combinazioni PCA + Clustering — LL globale (reale)

N. componenti	N. cluster	Devianza intra	Devianza persa
3	15	0.0155	0.0566
3	10	0.0253	0.0965
3	5	0.0706	0.2110

Nota: tabella parziale, riportate le configurazioni più significative. La combinazione a 3 componenti e 15 cluster minimizza la devianza persa.

**Figura 3.** Dendrogramma — LL globale, 3 componenti, 15 cluster (reale).

3.4. Clustering e scalabilità

3.4.1. Il problema della scalabilità

A differenza dei dataset LL (dell'ordine di poche migliaia di osservazioni), il dataset HL raccolto durante la sollecitazione reale conta 679 046 righe. L'approccio di clustering gerarchico agglomerativo utilizzato per i dataset LL richiede il calcolo della matrice delle distanze tra tutte le coppie di osservazioni, con complessità $O(n^2)$ sia in tempo che in memoria: per $n = 679\,046$, ciò comporterebbe circa 2.3×10^{11} coppie, corrispondenti a diverse centinaia di gigabyte di memoria — un requisito non soddisfacibile dall'hardware utilizzato (Tabella 2).

Si rende quindi necessario un approccio alternativo, in grado di elaborare l'intero dataset senza mai caricarlo integralmente in memoria né calcolarne la matrice delle distanze complete.

3.4.2. Pipeline incrementale a blocchi (chunking)

La soluzione adottata scompone il dataset in blocchi (*chunk*) di dimensione fissa, elaborati sequenzialmente in più passate sul file sorgente. La pipeline si articola in cinque fasi:

1. **Standardizzazione incrementale:** le statistiche di media e deviazione standard necessarie alla standardizzazione delle feature vengono calcolate incrementalmente su blocchi successivi, tramite un accumulatore (`StandardScaler.partial_fit`), senza mai richiedere l'intero dataset in memoria.
2. **PCA incrementale:** analogamente, la riduzione dimensionale viene effettuata tramite `IncrementalPCA`, che aggiorna le componenti principali blocco per blocco anziché tramite un'unica decomposizione in valori singolari sull'intera matrice.
3. **Costruzione del CF-tree (Birch):** le osservazioni proiettate nello spazio delle componenti principali vengono aggregate incrementalmente in una struttura ad albero compatta (*Clustering Feature tree*), secondo l'algoritmo BIRCH [1], che riassume porzioni del dataset in *sub-cluster* rappresentati dal proprio centroide e dal numero di osservazioni associate.
4. **Clustering gerarchico sui sub-cluster:** poiché il numero di sub-cluster generati da BIRCH è ordini di grandezza inferiore al numero di osservazioni originali, diventa possibile applicare il clustering gerarchico agglomerativo (identico a quello utilizzato per i dataset LL) direttamente sui centroidi dei sub-cluster.
5. **Assegnazione finale:** in un'ultima passata sul dataset, ogni osservazione originale viene associata al proprio sub-cluster e, transitivamente, al cluster finale risultante dal passo precedente.

Questo approccio consente di derivare una caratterizzazione dell'intero dataset — non di un suo campione — pur mantenendo un costo computazionale gestibile, poiché il passaggio quadratico viene applicato non alle 679 046 osservazioni originali, ma ai soli sub-cluster generati da BIRCH.

3.4.3. Risultati

Applicando la pipeline al dataset HL (feature `elapsed`, `bytes`, `latency`), le prime 2 componenti principali spiegano il 99.93% della varianza totale, un valore più elevato rispetto ai dataset LL, coerentemente con la forte correlazione attesa tra tempo di risposta, byte trasferiti e latenza. La costruzione del CF-tree con soglia `threshold = 0.5` produce 153 sub-cluster, sui quali viene successivamente applicato il clustering gerarchico per ottenere 15 cluster finali, in linea con il numero di cluster scelto per i dataset LL. La devianza persa complessiva risulta pari a 0.0106.

Tabella 6. Risultati della pipeline incrementale — dataset HL (reale)

N. componenti	N. cluster	N. sub-cluster (BIRCH)	Devianza persa
2	15	153	0.0106

3.4.4. Caratterizzazione del workload sintetico tramite tabella di contingenza

A partire dall'assegnazione di cluster ottenuta per ciascuna delle 679 046 richieste, viene costruita una tabella di contingenza tra cluster e risorsa richiesta (`label`), al fine di identificare, per ciascun cluster, la risorsa più frequentemente richiesta. Dall'analisi emerge una distribuzione fortemente sbilanciata tra i cluster: i primi quattro cluster per dimensione coprono congiuntamente oltre il 99% delle osservazioni, mentre i restanti undici cluster contano da 1 a poche centinaia di osservazioni ciascuno. Tale sbilanciamento è coerente con la natura del CF-tree di BIRCH, che tende a isolare in sub-cluster di piccole dimensioni le osservazioni con comportamento anomalo (ad esempio dovuto a variabilità di rete), mentre concentra la maggioranza del traffico "regolare" in pochi cluster densi. I cluster con dimensione inferiore a una soglia minima (50 osservazioni) vengono pertanto esclusi dalla costruzione del workload sintetico, poiché rappresentano una frazione trascurabile del traffico complessivo (0.014%) e sono verosimilmente riconducibili a rumore statistico piuttosto che a pattern di utilizzo genuini.

3.5. Costruzione del workload sintetico

3.5.1. Pesatura delle risorse

A partire dalla tabella di contingenza cluster-risorsa descritta nella sezione precedente, e filtrati i cluster di dimensione inferiore alla soglia minima, si procede alla costruzione del workload sintetico. Per ciascun cluster sopravvissuto al filtraggio viene identificata la risorsa più frequentemente richiesta; qualora la medesima risorsa risulti la più frequente in più cluster distinti, le rispettive dimensioni vengono sommate, ottenendo un peso complessivo proporzionale alla frequenza osservata nel workload reale.

Il peso assegnato a ciascuna risorsa viene quindi normalizzato su un budget complessivo di 100 unità, con un vincolo di peso minimo pari a 1 per evitare che risorse comunque rilevanti, ma con frequenza relativa inferiore all'1%, vengano eliminate dall'arrotondamento.

Tabella 7. Pool pesata delle risorse per il workload sintetico

Risorsa	Dimensione cluster aggregata	Peso (%)
<code>Sample_13.5KB.bin</code>	485,444	71
<code>Sample_99.7KB.bin</code>	153,223	23
<code>Sample_9.21MB.bin</code>	33,547	5
<code>Sample_5.07MB.bin</code>	6,367	1
<code>Sample_98KB.bin</code>	372	1

Nota: i pesi relativi a `-NoValue-` e `-NoValue-`

sono stati vincolati al valore minimo di 1 dall'arrotondamento, pur rappresentando rispettivamente circa lo 0.94% e lo 0.05% del traffico osservato.

A differenza del workload reale, in cui le richieste sono distribuite su 24 risorse con pesi decrescenti per categoria dimensionale, il workload sintetico risulta così concentrato su un numero ridotto di risorse (5), fortemente sbilanciato verso la risorsa di dimensione minore, che da sola rappresenta il 71% del traffico generato. Tale concentrazione riflette direttamente la distribuzione osservata nei cluster del dataset HL reale.

3.5.2. Generazione del traffico sintetico

Il workload sintetico viene sollecitato sul medesimo Web Server, riutilizzando l'infrastruttura di raccolta dati (load generator e collector) già descritta, con la sola differenza che il load generator seleziona le risorse dalla pool pesata per cluster anziché da quella pesata per categoria dimensionale.

La forte concentrazione del traffico su una singola risorsa di piccola dimensione (13.5 KB), unita all'elevato numero di richieste comple-

tabili nell'unità di tempo, ha reso necessario introdurre un tempo di attesa (*think time*) di 10 ms tra una richiesta e la successiva per ciascun thread worker. In assenza di tale accorgimento, il throughput generato eccede la capacità del sistema operativo di gestire nuove connessioni di rete nell'unità di tempo (esaurimento dei *file descriptor* disponibili), causando un tasso di fallimento delle richieste non rappresentativo di un carico applicativo realistico, bensì di una condizione di saturazione del client di generazione del traffico.

Con tale accorgimento, la sollecitazione del workload sintetico (durata 3600 secondi, 15 thread worker) produce 1 224 394 richieste, con un tasso di successo del 100%.

3.6. Caratterizzazione LL (sintetico)

A seguito della sollecitazione del workload sintetico, vengono raccolti i report a basso livello $LL'_{globale}$ e $LL'_{processo}$, secondo le medesime modalità impiegate per il workload reale. Pulizia e caratterizzazione vengono condotte applicando la medesima metodologia descritta nelle sezioni precedenti, mantenendo invariati numero di componenti principali e numero di cluster rispetto al corrispettivo dataset reale, condizione necessaria per rendere valido il successivo confronto statistico.

3.6.1. Pulizia

L'ispezione delle distribuzioni campionarie conferma, anche per i dataset sintetici, la presenza di transienti di *ramp-up* e *cool-down* analoghi a quelli osservati nei dataset reali, sebbene di estensione leggermente diversa: 11 righe iniziali e 24 finali per $LL'_{globale}$, 9 righe iniziali e 20 finali per $LL'_{processo}$.

Si osserva inoltre, per il dataset $LL'_{globale}$, un numero di outlier interni al periodo di misurazione (3.40% delle osservazioni) superiore rispetto al corrispondente dataset reale. Tale differenza è coerente con la natura del workload sintetico: la forte concentrazione del traffico su risorse di dimensione molto eterogenea (da 13.5 KB a 9.21 MB) genera transienti di carico più marcati e ricorrenti, in particolare in corrispondenza delle richieste verso le risorse di dimensione maggiore. Tali osservazioni, non concentrate in un singolo intervallo temporale ma distribuite lungo l'intera durata della misurazione, vengono interpretate come variabilità legittima del workload sintetico e pertanto mantenute nel dataset.

Tabella 8. Sintesi delle operazioni di pulizia — dataset sintetici

Dataset	Righe (grezze → pulite)	Colonne rimosse	Trimming
$LL'_{globale}$	3615 → 3580	<code>mem_total</code>	11 / 24
$LL'_{processo}$	3615 → 3586	<code>pid</code> , I/O	9 / 20

3.6.2. PCA e Clustering

Applicando PCA e clustering gerarchico con i medesimi parametri selezionati per il workload reale (3 componenti e 15 cluster per $LL'_{globale}$; 2 componenti e 15 cluster per $LL'_{processo}$), si ottengono i risultati riportati in Tabella 9.

Tabella 9. Confronto della devianza persa tra workload reale e sintetico

Dataset	N. componenti / cluster	Devianza persa (reale)	Devianza persa (sintetico)
LL globale	3 / 15	0.0566	0.0214
LL processo	2 / 15	0.0609	0.0400

Nota: in entrambi i casi, la devianza persa risulta inferiore nel workload sintetico rispetto al reale.

In entrambi i dataset, la devianza persa risulta inferiore nel workload sintetico rispetto al corrispettivo reale. Tale risultato è atteso: essendo il workload sintetico costruito a partire da un numero ridotto

di risorse (5 anziché 24), il sistema sotto sollecitazione presenta un comportamento meno eterogeneo, e i cluster risultanti sono pertanto più compatti e meglio separati.

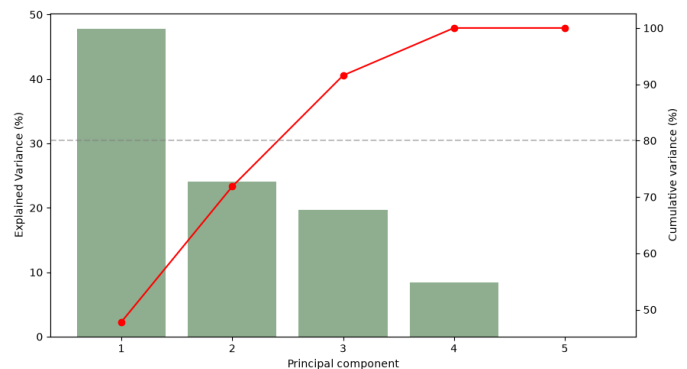


Figura 4. Scree plot delle componenti principali — LL globale (sintetico).

3.7. Validazione Statistica

3.7.1. Estrazione dei campioni per cluster

Al fine di confrontare statisticamente il workload reale e quello sintetico, per ciascun dataset (LL globale e LL di processo, nelle rispettive versioni reale e sintetica) viene estratto un campione rappresentativo per ciascuno dei 15 cluster individuati, selezionando l'osservazione più vicina al centroide del proprio cluster nello spazio delle componenti principali. Tale criterio, tra quelli possibili (selezione casuale, centroide stesso, osservazione più vicina al centroide), viene scelto in quanto privilegia elementi realmente rappresentativi del comportamento tipico di ciascun cluster, coerentemente con l'obiettivo di confrontare le caratteristiche tipiche dei due workload.

Si ottengono così due coppie di campioni ridotti, ciascuna composta da 15 osservazioni (una per cluster) nello spazio delle rispettive componenti principali: una coppia a 3 componenti per il confronto LL globale, una coppia a 2 componenti per il confronto LL di processo.

3.7.2. Test di normalità

Viene preliminarmente valutata la normalità delle distribuzioni tramite test di Kolmogorov-Smirnov, con ipotesi nulla H_0 : il campione proviene da una distribuzione normale standard.

Tabella 10. Test di Kolmogorov-Smirnov sulle componenti principali

Dataset	Componente	Statistica KS	p-value	Rigetta H_0
LL globale	PC1	0.1753	0.6828	No
LL globale	PC2	0.1875	0.6022	No
LL globale	PC3	0.2733	0.1759	No
LL processo	PC1	0.1213	0.9607	No
LL processo	PC2	0.1019	0.9931	No

Nota: risultati identici tra campione reale e sintetico in ciascuna componente.

Nel caso in questione nessuna componente rigetta H_0 . Tale esito va tuttavia interpretato con cautela: la dimensione campionaria estremamente ridotta ($n = 15$, una osservazione per cluster) conferisce al test una potenza statistica limitata, insufficiente a discriminare con sicurezza tra distribuzioni normali e non normali. Il mancato rigetto non costituisce pertanto una prova di normalità, ma l'assenza di evidenza sufficiente contro di essa a questa dimensione campionaria.

Per tale motivo, si procede comunque con un test non parametrico, più robusto rispetto ad assunzioni di normalità non verificabili con certezza sul campione a disposizione.

3.7.3. Test di Wilcoxon

Viene applicato il test di Wilcoxon (*ranksum*), con ipotesi nulla H_0 : i due campioni (reale e sintetico) provengono da distribuzioni con la medesima mediana, componente per componente.

Tabella 11. Test di Wilcoxon ranksum — confronto reale vs. sintetico

Dataset	Componente	Statistica	p-value	Rigetta H_0
LL globale	PC1	−1.1406	0.2540	No
LL globale	PC2	1.8043	0.0712	No
LL globale	PC3	0.8503	0.3952	No
LL processo	PC1	0.4355	0.6632	No
LL processo	PC2	1.0577	0.2902	No

In nessuna delle componenti analizzate, per entrambi i dataset, il test rigetta l’ipotesi nulla a un livello di significatività del 5%. Si può pertanto concludere che non vi è evidenza statistica di una differenza tra le mediane dei workload reale e sintetico, nello spazio delle componenti principali considerate.

3.8. Conclusioni

Il presente capitolo ha descritto la costruzione di una pipeline completa per la caratterizzazione di un workload applicativo, articolata secondo lo schema a tre fasi: caratterizzazione del workload reale, costruzione e sollecitazione di un workload sintetico, validazione statistica della loro equivalenza.

L’intera pipeline è stata implementata in Python, con l’obiettivo di ottenere uno strumento modulare e riproducibile. Tale scelta ha richiesto di affrontare, rispetto all’approccio originale, alcune problematiche specifiche: in primo luogo, la scalabilità del clustering gerarchico rispetto alla dimensione del dataset HL (679 046 osservazioni), risolta tramite una pipeline incrementale a blocchi basata sull’algoritmo BIRCH; in secondo luogo, la gestione di un ambiente di raccolta dati su macchina singola, condivisa tra generazione del carico e misurazione delle risorse di sistema, che ha richiesto accorgimenti specifici (tempo di attesa tra le richieste, gestione dei limiti di sistema) per evitare che il processo di misurazione stesso alterasse il fenomeno osservato.

I risultati ottenuti mostrano che, per entrambi i dataset a basso livello analizzati (metriche globali di sistema e metriche del processo server), il workload sintetico costruito a partire dalla caratterizzazione del traffico applicativo (HL) non presenta differenze statisticamente significative rispetto al workload reale, nello spazio delle rispettive componenti principali. Tale risultato conferma, nei limiti del test di Wilcoxon applicato a un campione di ridotta numerosità, la validità dell’approccio PCA + Clustering come metodologia per la sintesi di workload rappresentativi a partire da misurazioni applicative ad alto livello.

Un limite dell’approccio adottato risiede proprio nella dimensione del campione utilizzato per la validazione statistica finale ($n = 15$, pari al numero di cluster), che ne limita la potenza statistica. Un possibile sviluppo futuro potrebbe prevedere l’estrazione di più osservazioni rappresentative per cluster, o l’impiego di test statistici progettati per campioni di dimensione ridotta, al fine di rafforzare la robustezza della conclusione raggiunta.

4. WORKLOAD CHARACTERIZATION

I
5. DOE

I
6. REGRESSION

I

7. BENCHMARK

I
8. FFDA

I
9. RDB E FAULT TREE

I
■ RIFERIMENTI BIBLIOGRAFICI

[1] T. Zhang, R. Ramakrishnan e M. Livny, «BIRCH: an efficient data clustering method for very large databases,» *SIGMOD Rec.*, vol. 25, n. 2, pp. 103–114, giu. 1996, ISSN: 0163-5808. DOI: [10.1145/235968.233324](https://doi.org/10.1145/235968.233324) indirizzo: <https://doi.org/10.1145/235968.233324>